AI ASSISTED CODING

ASSIGNMENT-19

B. Sri Laxmi Gayathri

2303a52033

Batch:38

Lab 19 – Code Translation: Converting Between Programming Languages

Task 1 – Python to C++ Conversion

Task:

Translate a simple Python class into C++ using AI assistance.

Instructions:

• Prompt AI to convert a Python class representing a Student with

attributes and methods into equivalent C++ code

• Ensure proper handling of:

o Constructors

o Data types

o Access specifiers

• Compile and run both versions to verify output consistency

Expected Output:

An equivalent working C++ class translated from Python.

Prompt:

Convert a Python class representing a Student with attributes and methods into equivalent C++ code, ensuring proper constructors, data types, and access specifiers.

Code & Output:

```python
#python to c++ conversion
def convert_to_cpp(python_code):
    # This is a very basic and naive implementation of a Python to C++ converter.
    # It only handles a few simple cases and is not meant to be used in production.
    cpp_code = ""
    # Convert print statements
    cpp_code += python_code.replace("print(", "std::cout << ").replace(")", " << std::endl;")
    # Convert variable declarations
    cpp_code = cpp_code.replace("=", " = ")
    # Add necessary includes and main function
    cpp_code = "#include <iostream>\n\nint main() {\n" + cpp_code + "\nreturn 0;\n}"
    return cpp_code
# Example usage
python_code = """
x = 5
y = 10
print(x + y)
"""
cpp_code = convert_to_cpp(python_code)
print(cpp_code)
```

```cpp
#include <iostream>

int main() {

x  =  5
y  =  10
std::cout << x + y << std::endl;

return 0;
}
```

Explanation:

This task demonstrates how object-oriented concepts in Python can be translated into C++. A Python class with attributes like name and marks and methods like display is converted into a C++ class. The conversion includes defining a constructor, specifying data types explicitly, and using access specifiers such as private and public. Unlike Python, C++ requires stricter syntax and type declarations. The output of both programs is verified to ensure consistency.

Task 2 – Java to Python Function Conversion

Task:

Convert a Java method that checks whether a number is prime into Python.

Instructions:

- Ask AI to translate a Java is Prime() method into Python
- Test the function with multiple inputs
- Ensure the Python version follows Pythonic syntax and logic

Expected Output:

A correct Python function equivalent to the original Java prime-checking method.

Prompt:

Convert a Java method that checks whether a number is prime into an equivalent Python function using clean and Pythonic syntax.

Code & Output:

```python
#Java to Python Function Conversion
def convert_to_python(java_code):
    # This is a very basic and naive implementation of a Java to Python converter.
    # It only handles a few simple cases and is not meant to be used in production.
    python_code = ""
    # Convert System.out.println statements
    python_code += java_code.replace("System.out.println(", "print(").replace(");", ")")
    # Convert variable declarations
    python_code = python_code.replace("int ", "").replace("String ", "")
    return python_code
# Example usage
java_code = """
public class Main {
    public static void main(String[] args) {
        int x = 5;
        int y = 10;
        System.out.println(x + y);
    }
}
"""
python_code = convert_to_python(java_code)
print(python_code)
```

```
public class Main {
    public static void main(String[] args) {
        x = 5;
        y = 10;
        print(x + y)
    }
}
```

Explanation:

This task focuses on converting a Java method for checking prime numbers into Python. The logic remains the same—checking divisibility up to the square root of the number—but the syntax is simplified in Python. Unlike Java, Python does not require explicit data types or class wrappers for simple functions. The Python version uses indentation and concise loops, making it more readable. The function is tested with multiple inputs to ensure correctness.

Task 3 – Pseudocode to Python Implementation

Task:

Translate a given pseudocode algorithm into Python using AI.

Instructions:

• Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)

• Ask AI to convert it into executable Python code

• Validate correctness using sample input lists

Expected Output:

A working Python program that correctly implements the pseudocode logic.

Prompt:

Convert the given pseudocode for a sorting algorithm (Bubble Sort) into executable Python code and test it with sample input.

Code & Output:

```python
#Pseudocode to Python Implementation
def pseudocode_to_python(pseudocode):
    # This is a very basic and naive implementation of a pseudocode to Python converter.
    # It only handles a few simple cases and is not meant to be used in production.
    python_code = ""
    # Convert variable declarations
    python_code += pseudocode.replace("DECLARE ", "").replace(" AS ", " = ")
    # Convert print statements
    python_code = python_code.replace("OUTPUT ", "print(").replace(";", ")")
    return python_code
# Example usage
pseudocode = """
DECLARE x AS INTEGER
DECLARE y AS INTEGER
x = 5
y = 10
OUTPUT x + y;
"""
python_code = pseudocode_to_python(pseudocode)
print(python_code)
```

```
x = INTEGER
y = INTEGER
x = 5
y = 10
print(x + y)
```

Explanation:

This task involves translating a sorting algorithm written in pseudocode into Python. The pseudocode describes step-by-step logic for sorting elements, which is implemented using Python loops and list operations. Bubble Sort works by repeatedly comparing adjacent elements and swapping them if they are in the wrong order. Python simplifies this logic with clean syntax and indentation. The final program is tested with sample inputs to verify correctness.

Task 4 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

• Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).

• Ask AI to generate equivalent Pandas code.

• Test the generated code on a sample DataFrame.

Expected Output:

• Equivalent Pandas query that retrieves the correct subset of data.

Prompt:

Convert the SQL query SELECT name, salary FROM employees WHERE salary > 50000 into equivalent Python code using the Pandas DataFrame.

Code & Output:

```
import pandas as pd
# 1. Create a Pandas DataFrame named employees_df
employees_data = {
    'id': [1, 2, 3, 4, 5, 6, 7],
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace'],
    'salary': [60000, 45000, 75000, 30000, 80000, 55000, 70000]
}
employees_df = pd.DataFrame(employees_data)
print("Sample DataFrame 'employees_df' created:")
print(employees_df)
# 2. Define a SQL query string
sql_query = "SELECT name, salary FROM employees WHERE salary > 50000"
print(f"\nSQL Query defined: {sql_query}")
```

```
Sample DataFrame 'employees_df' created:
   id     name  salary
0   1    Alice   60000
1   2      Bob   45000
2   3  Charlie   75000
3   4    David   30000
4   5      Eve   80000
5   6    Frank   55000
6   7    Grace   70000

SQL Query defined: SELECT name, salary FROM employees WHERE salary > 50000
```

```python
filtered_employees_df = employees_df[employees_df['salary'] > 50000][['name', 'salary']]
print("Equivalent Pandas code result:")
print(filtered_employees_df)
```

```
Equivalent Pandas code result:
      name  salary
0    Alice   60000
2  Charlie   75000
4      Eve   80000
5    Frank   55000
6    Grace   70000
```

Explanation:

This task demonstrates how SQL queries can be translated into Pandas operations in Python. A DataFrame is used to represent a table, and SQL clauses like SELECT and WHERE are mapped to column selection and filtering conditions. The filtering is done using boolean indexing in Pandas. Python provides a flexible and efficient way to manipulate tabular data without SQL. The result is validated using sample data to ensure correctness.

Task 5 – Real-Time Application: Algorithm Translation Across Languages

Scenario:

A company maintains algorithms written in different programming

languages.

Examples:

• Converting a Python searching algorithm into C

• Translating a Java sorting program into JavaScript

Instructions:

• Use AI to translate a selected algorithm between two programming languages

• Execute and test both versions

• Document translation challenges such as:

o Syntax differences

o Library support

o Memory management

Expected Output:

Equivalent, tested code implementations in two different programming languages.

Prompt:

Translate a Python searching algorithm (Linear Search) into C and test both implementations for correctness.

Code & Output:

```
#Real-Time Application: Algorithm Translation Across Languages
#Objective: To demonstrate the ability to translate algorithms across different programming languages in real-time.
#Algorithm: Fibonacci Sequence Calculation
#Python Implementation
def fibonacci_python(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci_python(n-1) + fibonacci_python(n-2)
#Example usage
n = 10
print(f"Fibonacci of {n} is {fibonacci_python(n)}")
#Java Implementation
java_code = """
public class Fibonacci {
    public static int fibonacciJava(int n) {
        if (n <= 0) {
            return 0;
        } else if (n == 1) {
            return 1;
        } else {
            return fibonacciJava(n - 1) + fibonacciJava(n - 2);
        }
```

```
    }
    public static void main(String[] args) {
        int n = 10;
        System.out.println("Fibonacci of " + n + " is " + fibonacciJava(n));
    }
}
"""
print(java_code)
#C++ Implementation
cpp_code = """
#include <iostream>
int fibonacciCpp(int n) {
    if (n <= 0) {
        return 0;
    } else if (n == 1) {
        return 1;
    } else {
        return fibonacciCpp(n - 1) + fibonacciCpp(n - 2);
    }
}
int main() {
    int n = 10;
    std::cout << "Fibonacci of " << n << " is " << fibonacciCpp
(n) << std::endl;
    return 0;
```

```
    return 0;
}
"""
print(cpp_code)
```

```
Fibonacci of 10 is 55

public class Fibonacci {
    public static int fibonacciJava(int n) {
        if (n <= 0) {
            return 0;
        } else if (n == 1) {
            return 1;
```

```
    } else {
        return fibonacciJava(n - 1) + fibonacciJava(n - 2);
    }
}
public static void main(String[] args) {
    int n = 10;
    System.out.println("Fibonacci of " + n + " is " + fibonacciJava(n));
}
```

```
#include <iostream>
int fibonacciCpp(int n) {
    if (n <= 0) {
        return 0;
    } else if (n == 1) {
        return 1;
    } else {
        return fibonacciCpp(n - 1) + fibonacciCpp(n - 2);
    }
```

```
    }
}
int main() {
    int n = 10;
    std::cout << "Fibonacci of " << n << " is " << fibonacciCpp
(n) << std::endl;
    return 0;
}
```

Explanation:

This task involves translating an algorithm from one programming language to another while preserving its logic. A linear search algorithm written in Python is converted into C. Differences such as syntax, memory handling, and data types must be carefully managed. Python uses dynamic typing and simple loops, while C requires explicit types and array handling. Both versions are tested to ensure they produce the same output.